ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026



Khoảng 25 giây. Chào thầy và các bạn. Nhóm 4 trình bày ConfigStream, hệ thống lưu trữ và phân tích thay đổi cấu hình. Buổi checkpoint này tập trung vào bài toán, dữ liệu, kiến trúc và những phần nhóm đã triển khai được. Chưa trình bày như một sản phẩm đã hoàn thiện hoặc đã chứng minh khả năng mở rộng.

The problem: configuration changes over time

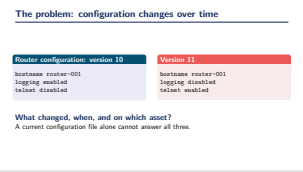
Router configuration: version 10

```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?
A current configuration file alone cannot answer all three.



Khoảng 45 giây. Mở đầu bằng ví dụ cụ thể: cùng một router nhưng phiên bản sau tắt logging và bật telnet. Đây là dữ liệu minh hoạ do nhóm tạo, không phải sự cố thực tế. Nếu chỉ giữ file hiện tại, ta không biết thay đổi xảy ra lúc nào và khác phiên bản trước ra sao. Nhóm muốn lưu lịch sử và chỉ ra phần thay đổi thay vì chỉ giữ trạng thái cuối cùng.

Four datasets, four meanings of change

Terraform / TerraDS: versions

Infrastructure code stored in Git.

v1 → **v2**: which lines changed?

RPKI: observed daily states

Observed route-authorization records.

Day 1 → **day 2**: which members differ?

RIS: routing events

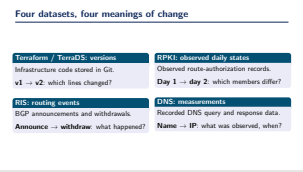
BGP announcements and withdrawals.

Announce → **withdraw**: what happened?

DNS: measurements

Recorded DNS query and response data.

Name → **IP**: what was observed, when?



Khoảng 65 giây. Nhóm dùng bốn loại dữ liệu có ý nghĩa thời gian khác nhau. Terraform từ TerraDS là các phiên bản file mã hạ tầng trong Git: so sánh để biết dòng nào đổi, không coi commit là lần triển khai. RPKI là tập bản ghi uỷ quyền định tuyến quan sát theo ngày; so sánh thành viên giữa hai ngày đầy đủ, không suy ra chính xác thao tác của quản trị viên. RIS ghi nhận sự kiện BGP, ví dụ quảng bá hoặc rút tuyến; hai sự kiện giống nội dung vẫn có thể là hai lần xảy ra khác nhau. DNS là phép đo truy vấn và phản hồi: thấy tên miền trả về địa chỉ nào tại thời điểm quan sát, không coi đó là toàn bộ trạng thái DNS. Ví dụ trên slide chỉ minh hoạ cách hiểu. Điểm chung là cần lưu lịch sử; cách định nghĩa thay đổi phải riêng cho từng loại.

Prepared data: accepted scope for each dataset

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

Controlled synthetic inputs also test retries and late arrivals.
Source manifests and result receipts are versioned in Git.

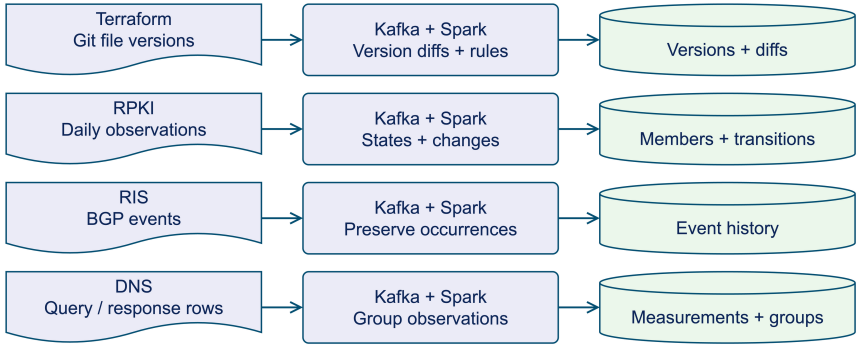
Prepared data: accepted scope for each dataset

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

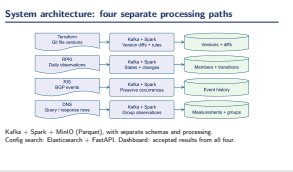
Controlled synthetic inputs also test retries and late arrivals.
Source manifests and result receipts are versioned in Git.

Khoảng 55 giây. Đây là phạm vi đã được kiểm chứng trong các lần chạy ghi nhận, không phải tổng kích thước mọi nguồn ngoài Internet và không phải lần chạy mới hôm nay. Terraform có 10.188 phiên bản của 6.505 lịch sử file thuộc 375 repo. RPKI có 14 ngày đầy đủ trong phạm vi ripencc.tal; số 14 là số ngày quan sát, không phải số bản ghi bên trong. RIS có khoảng 4,73 triệu lần xảy ra sự kiện trong một giờ ở collector rrc00. DNS có khoảng 2,23 triệu dòng phép đo từ hai đối tượng nguồn .li đã chọn. Không cộng bốn con số vì đơn vị khác nhau. Nhóm còn có generator để chủ động kiểm thử dữ liệu lặp và đến muộn. Dữ liệu lớn lưu ngoài Git; repo giữ manifest và bằng chứng kết quả.

System architecture: four separate processing paths

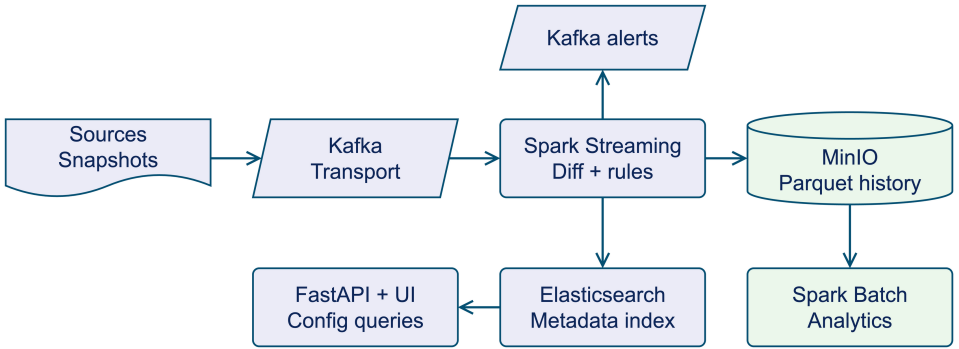


Kafka + Spark + MinIO (Parquet), with separate schemas and processing.
Config search: Elasticsearch + FastAPI. Dashboard: accepted results from all four.

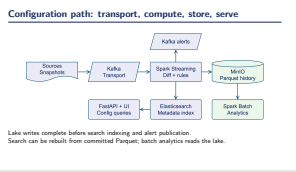


Khoảng 60 giây. Bốn hàng là bốn đường xử lý đã có trong repo, không phải tất cả đều đi qua bộ diff cấu hình. Hàng Terraform giữ logic phiên bản và rule. RPKI xử lý tập thành viên của các ngày đầy đủ và thay đổi giữa chúng. RIS giữ các lần xảy ra sự kiện, chỉ hoà giải bản gửi lại cùng danh tính nguồn; không xoá sự kiện chỉ vì nội dung giống nhau. DNS giữ phép đo và nhóm kết quả có thông tin nguồn để so sánh mô tả. Các luồng dùng chung công nghệ Kafka, Spark và lưu Parquet trên MinIO nhưng có schema, topic và metadata commit riêng. Elasticsearch và FastAPI phục vụ đường cấu hình; dashboard hiện có bản tổng hợp bằng chứng của cả bốn miền, không phải API live chung cho mọi miền. Dữ liệu thực ở đây được phát lại từ các nguồn đã cố định.

Configuration path: transport, compute, store, serve



Lake writes complete before search indexing and alert publication.
Search can be rebuilt from committed Parquet; batch analytics reads the lake.



Khoảng 50 giây. Đi theo hàng chính từ trái sang phải: nguồn gửi snapshot qua Kafka, Spark xử lý state, diff và rule, rồi ghi vào MinIO. Nhánh dưới bên phải là Spark batch đọc lịch sử đã commit để phân tích. Nhánh search giữ metadata phục vụ FastAPI và giao diện, không chứa toàn bộ nội dung cấu hình. Nhánh trên là cảnh báo Kafka. Các mũi tên biểu diễn đường dữ liệu; thứ tự thực thi trong callback là lake trước, search sau, rồi mới phát alerts. Khi index mất hoặc cần tạo lại, có lệnh reindex đọc committed Parquet. Đây là cấu hình bật lake và search, không nói mọi chế độ chạy đều bắt buộc có tất cả thành phần.

Input contract: identity, source version and two clocks

Field	Illustrative value	Role in processing
`asset_id`	`router-001`	Groups one configuration history
`snapshot_id`	`router-001-v11`	Detects conflicting reuse of an ID
`version`	11	Selects the adjacent predecessor, v10
`event_time`	08:00 UTC	Source time for watermark and timers
`ingest_time`	08:03 UTC	Collector acceptance time
`hash`	SHA-256 of content	Checks inline content integrity

The schema accepts exactly one of `content` or `content_uri`.
The current diff processor handles inline text; URI content is not fetched.

Input contract: identity, source version and two clocks

Field	Illustrative value	Role in processing
asset_id	router-001	Groups one configuration history
snapshot_id	router-001-v11	Detects conflicting reuse of an ID
version	11	Selects the adjacent predecessor, v10
event_time	08:00 UTC	Source time for watermark and timers
ingest_time	08:03 UTC	Collector acceptance time
hash	SHA-256 of content	Checks inline content integrity

The schema accepts exactly one of content or content_uri.
The current diff processor handles inline text; URI content is not fetched.

Khoảng 45 giây. Một message không chỉ có nội dung file mà phải có danh tính, phiên bản và thời gian. Asset ID gom lịch sử của một đối tượng; snapshot ID xác định một bản ghi nguồn. Version mới là cơ sở chọn cặp liền kề, không dùng offset hoặc thời gian nhận. Event time là thời điểm nguồn quan sát, còn ingest time là lúc collector nhận. Ví dụ lệch ba phút chỉ minh hoạ hai đồng hồ. SHA-256 kiểm tra nội dung inline; đây không phải chữ ký xác thực nguồn. Schema cho phép content hoặc URI nhưng processor diff hiện chỉ xử lý văn bản inline, không tự tải file từ URI.

Kafka: delivery position differs from source version

Three input topics

`config.assets`: metadata

`config.events`: change audit

`config.snapshots`: content

Key: `asset_id`

Spark processes the snapshots topic.

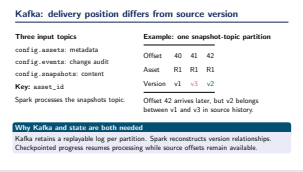
Example: one snapshot-topic partition

Offset	40	41	42
Asset	R1	R1	R1
Version	v1	v3	v2

Offset 42 arrives later, but v2 belongs between v1 and v3 in source history.

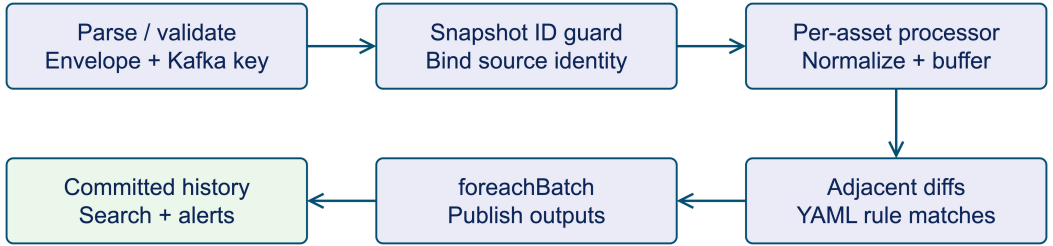
Why Kafka and state are both needed

Kafka retains a replayable log per partition. Spark reconstructs version relationships. Checkpointed progress resumes processing while source offsets remain available.



Khoảng 50 giây. Producer gửi metadata asset, sự kiện audit và snapshot vào ba topic riêng. Kafka key là asset ID; Spark xử lý topic snapshots và không chờ event ở topic khác đến trước. Bảng bên phải minh hoạ cùng asset: offset 40, 41, 42 mang version 1, 3, 2. Kafka vẫn giữ đúng thứ tự log nhưng lịch sử nguồn phải là 1, 2, 3. Kafka tách producer khỏi processor và giữ log để replay; Spark mới giải quyết quan hệ phiên bản. Checkpoint lưu tiến độ đã xử lý; nếu Kafka đã xoá offset do retention thì checkpoint không tự tạo lại dữ liệu.

Spark: two stateful stages before output publication

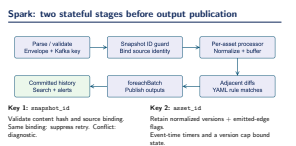


Key 1: `snapshot_id`

Validate content hash and source binding. Same binding: suppress retry. Conflict: diagnostic.

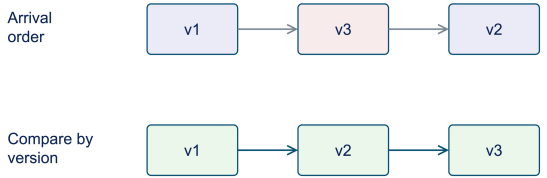
Key 2: `asset_id`

Retain normalized versions + emitted-edge flags.
Event-time timers and a version cap bound state.



Khoảng 65 giây. Đi theo sơ đồ từ parse tới output. SQL parse và kiểm tra envelope, Kafka key, thời gian. Sau đó có hai phép group và hai stateful operator native của Spark. Operator đầu group theo snapshot ID để kiểm tra binding về asset, version, thời gian nguồn, format, hash và URI. Binding giống nhau là retry; binding khác là conflict, kể cả khi cùng ID xuất hiện ở asset khác. Ingest time không thuộc danh tính nguồn. Operator sau mới group theo asset, chuẩn hoá nội dung, giữ buffer version và cờ cặp nào đã phát. Watermark và timer event time cùng giới hạn số version giữ bộ nhớ có giới hạn. Raw Kafka đi nhánh riêng nên bản ghi bị business logic từ chối vẫn còn bằng chứng. State được Spark checkpoint, không chỉ là dictionary Python trên driver.

Version state: buffer gaps and emit each adjacent pair



Input	Retained versions	New output
v1	{1}	Baseline only; no predecessor
v3	{1, 3}	Hold the gap; no 1 → 3 comparison
v2	{1, 2, 3}	Emit 1 → 2 and 2 → 3; mark both edges
Retry v2	Unchanged	No new logical diff while identity/state is retained

Version state: buffer gaps and emit each adjacent pair

Khoảng 60 giây. Đây là một trace nhỏ của thuật toán, không chỉ là hình sắp xếp. Khi v1 đến, lưu baseline. V3 đến nhưng thiếu v2 thì giữ khoảng trống và chưa tạo diff 1 sang 3. Khi v2 đến trong thời hạn còn giữ state, tìm được cả hai hàng xóm nên tạo 1 sang 2 và 2 sang 3. Cờ emitted-edge ghi nhận hai cặp đã phát. Nếu gửi lại v2 với cùng binding trong state còn giữ thì không tạo diff mới. Nếu dữ liệu quá muộn hoặc version đã bị eviction thì không hứa phục hồi online vô hạn; raw vẫn là bằng chứng của những record đã fetch. Timer tiến theo watermark event time, không đơn thuần theo đồng hồ treo tường.

From normalized text to an explainable rule alert

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Normalization handles selected text noise and masks supported secret patterns.

Actual YAML rule

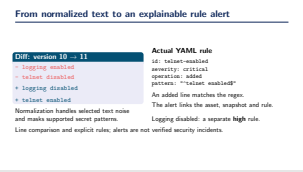
```
id: telnet-enabled
severity: critical
operation: added
pattern: "^telnet enabled$"
```

An added line matches the regex.

The alert links the asset, snapshot and rule.

Logging disabled: a separate **high** rule.

Line comparison and explicit rules; alerts are not verified security incidents.



Khoảng 50 giây. Chuẩn hoá thực hiện trước diff: xử lý một số nhiễu văn bản và che các mẫu secret được hỗ trợ, không sửa raw gốc. Bên trái là bốn dòng thay đổi của ví dụ đầu bài. Bên phải là các trường thật của rule YAML: chỉ xét dòng được thêm, match chính xác telnet enabled và tạo alert critical. Alert mang asset ID, snapshot ID, rule ID để truy ngược bằng chứng. Logging bị tắt được rule khác báo high. Logic nằm trong package processing dùng lại cho Spark và demo nhỏ; API không tính lại. Đây là so sánh dòng và rule tường minh, không phải parser hiểu mọi ngữ nghĩa cấu hình hoặc mô hình AI.

MinIO: raw evidence and typed Parquet history

```
config-lake/
  raw/snapshots/
  processed/snapshots/
  processed/diffs/
  processed/alerts/
  processed/diagnostics/
  _commits/
  analytics/
```

Raw: binary key/value plus Kafka topic, partition, offset and timestamp.

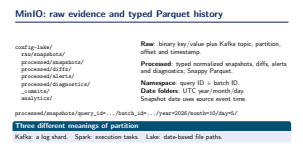
Processed: typed normalized snapshots, diffs, alerts and diagnostics; Snappy Parquet.

Namespace: query ID + batch ID.
Date folders: UTC year/month/day.
Snapshot date uses source event time.

```
processed/snapshots/query_id=.../batch_id=.../year=2026/month=10/day=5/
```

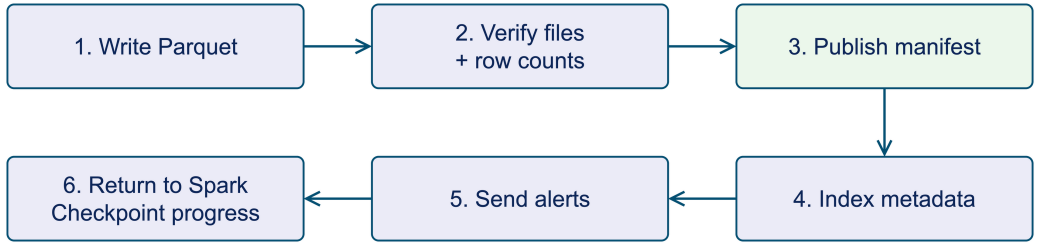
Three different meanings of partition

Kafka: a log shard. Spark: execution tasks. Lake: date-based file paths.



Khoảng 55 giây. MinIO cung cấp object storage tương thích S3, Parquet là định dạng file có schema theo cột. Raw giữ byte và provenance của record Kafka được fetch, processed giữ snapshot đã chuẩn hoá cùng diff, alert và diagnostic. Query ID và batch ID chia vùng ghi để nhận diện lần retry; date folder dùng năm tháng ngày UTC, riêng snapshot lấy event time. Không partition theo asset ID. Ba chữ partition rất dễ lẫn: Kafka là shard log, Spark là phân chia task khi thực thi, lake là tổ chức file theo ngày. Thiết kế thư mục không tự chứng minh query pruning hay tăng tốc; cần đo reader thực tế. Chỉ các file được manifest commit khai báo mới là lịch sử được chấp nhận.

Commit protocol: publish verified history before serving

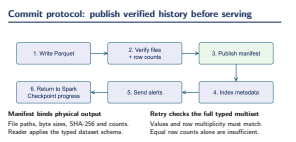


Manifest binds physical output

File paths, byte sizes, SHA-256 and counts.
Reader applies the typed dataset schema.

Retry checks the full typed multiset

Values and row multiplicity must match.
Equal row counts alone are insufficient.



Khoảng 55 giây. Một batch đi qua sáu bước. Trước hết ghi Parquet rồi kiểm tra inventory file, kích thước, SHA-256, schema và row count. Sau đó publish manifest để reader biết tập file nào hoàn tất. Search và Kafka alerts chạy sau commit lake; callback thành công mới cho Spark tiến hành checkpoint progress. Khi retry batch đã commit, không chỉ so số lượng dòng mà so toàn bộ projection có kiểu dữ liệu theo multiset cả hai chiều, bao gồm số lần mỗi dòng xuất hiện. Nếu khớp mới tái sử dụng output; khác thì fail. Bảo đảm này không biến ba hệ thống MinIO, Elasticsearch và Kafka thành một transaction atomic.

Failure recovery: what happens if a batch stops midway?

Failure point	Visible state	Retry behavior
Before lake manifest	Partial files are not committed history	Readers exclude the incomplete batch
After commit, during search	Lake is valid; search may be incomplete	Verify/reuse lake; index deterministic document IDs again
After alerts, before progress commit	Some alerts may already be delivered	Reprocessing may append physical alert duplicates

Spark checkpoint: source progress and operator state.

Lake manifest: which verified data files are committed.

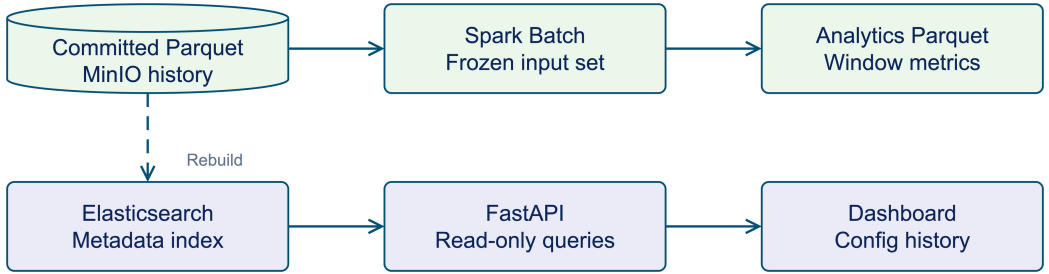
Failure recovery: what happens if a batch stops midway?

Failure point	Visible state	Retry behavior
Before lake manifest	Partial files are not committed history	Readers exclude the incomplete batch
After commit, during search	Lake is valid; search may be incomplete	Verify/reuse lake; index deterministic document IDs again
After alerts, before progress commit	Some alerts may already be delivered	Reprocessing may append physical alert duplicates

Spark checkpoint: source progress and operator state.
Lake manifest: which verified data files are committed.

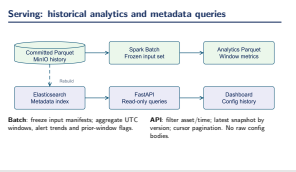
Khoảng 50 giây. Ba tình huống giúp giải thích cơ chế recovery cụ thể. Lỗi trước manifest thì file có thể tồn tại nhưng reader không coi là lịch sử hoàn tất. Lỗi ở Elasticsearch sau khi lake commit thì batch Spark thất bại; khi retry, code kiểm tra và dùng lại lake, rồi index lại bằng deterministic document ID. Nếu alerts đã đến Kafka mà tiến độ batch chưa commit, retry có thể gửi lại alert vật lý. Vì vậy không gọi toàn hệ thống exactly once. Cần phân biệt hai marker: checkpoint thuộc Spark để tiếp tục source progress và state, manifest thuộc lake để công bố file hoàn tất. Đây là hành vi code; lần recovery Kubernetes trên image integrity mới vẫn chưa hoàn tất, không lấy kết quả image cũ thay cho nó.

Serving: historical analytics and metadata queries



Batch: freeze input manifests; aggregate UTC windows, alert trends and prior-window flags.

API: filter asset/time; latest snapshot by version; cursor pagination. No raw config bodies.



Khoảng 60 giây. MinIO giữ lịch sử đã commit. Nhánh trên là Spark batch: chốt danh sách manifest đầu vào để commit mới không làm thay đổi dữ liệu giữa job, rồi tính tần suất thay đổi, xu hướng alert và ngưỡng bất thường theo các cửa sổ trước đó. Không dùng cửa sổ tương lai và không gọi đây là mô hình ML. Nhánh dưới là index Elasticsearch phục vụ truy vấn nhanh. Đường nét đứt từ lake là lệnh rebuild; khi chạy bình thường streaming index sau lake commit. FastAPI chỉ đọc metadata với filter asset, thời gian, latest theo version và cursor phân trang; không chạy Spark hoặc tính lại diff, không trả raw content. Dashboard research của bốn miền dùng bundle kết quả đã chấp nhận, khác với history gọi API.

Progress: runnable code and a recorded pipeline replay

Repository

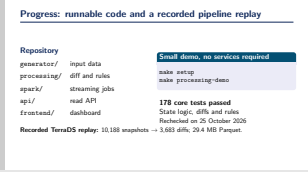
`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

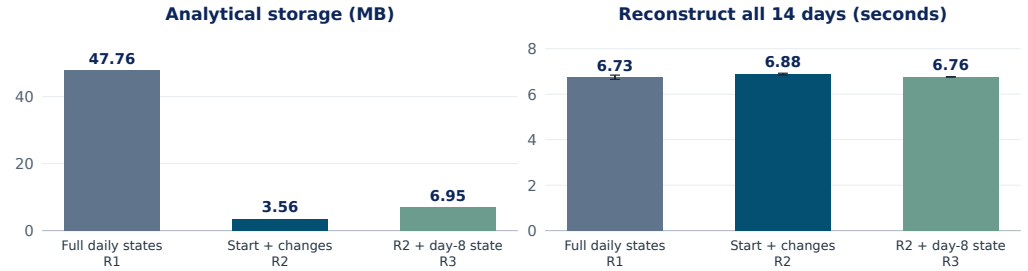
Recorded TerraDS replay: 10,188 snapshots → 3,683 diffs; 29.4 MB Parquet.



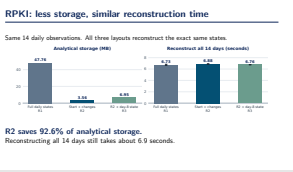
Khoảng 60 giây. Repo có các thành phần ở cột trái. Hai lệnh bên phải chạy demo Python không cần service; 178 test của state, diff và rule đã được kiểm tra khi chuẩn bị slide. Dòng dưới là minh chứng tích hợp đã lưu: TerraDS đi qua Kafka, Spark và MinIO với 10.188 snapshot, 3.683 cặp liền kề và 29.402.997 byte Parquet. Phân biệt rõ demo nhỏ có thể chạy ngay với replay lịch sử đã được ghi nhận, không gọi đây là lần replay mới hôm nay. Nếu thầy yêu cầu demo trực tiếp, chuyển sang terminal đã chuẩn bị; không dùng kết quả này để suy ra HA hay scalability.

RPKI: less storage, similar reconstruction time

Same 14 daily observations. All three layouts reconstruct the exact same states.



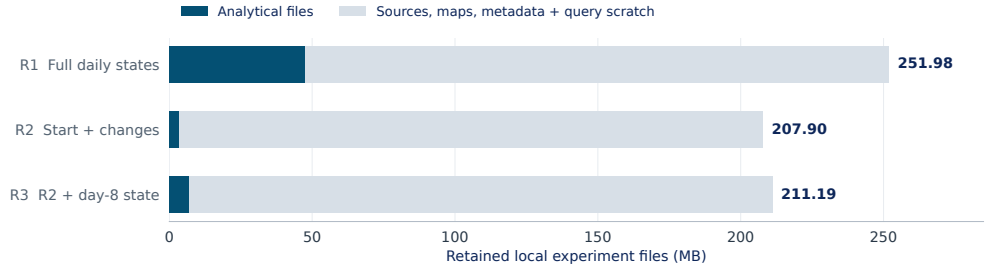
R2 saves 92.6% of analytical storage.
Reconstructing all 14 days still takes about 6.9 seconds.



Khoảng 60 giây. Đây là phép so sánh trên cùng 14 ngày RPKI đã quan sát, không so số lượng bản ghi giữa bốn miền. R1 lưu toàn bộ tập thành viên của mỗi ngày. R2 lưu ngày đầu và các tập thêm, bớt giữa các ngày. R3 giống R2 nhưng thêm trạng thái đầy đủ ở ngày 8. Cả ba cùng giữ các transition sidecar chung, dùng cùng encoding và Snappy; đây là layout thí nghiệm local, chưa thay định dạng production. Biểu đồ trái: dung lượng phân tích giảm từ 47,76 MB xuống 3,56 MB ở R2, tức giảm 92,6 phần trăm. Biểu đồ phải: dựng lại đủ 14 ngày mất 6,73 giây ở R1 và 6,88 giây ở R2, gần tương đương trong phép đo này. R3 nằm giữa về dung lượng. Tất cả trạng thái và kết quả truy vấn khớp chính xác, không chỉ khớp số lượng. Kết luận: lưu thay đổi là ứng viên tốt cho lịch sử RPKI đã thử. Đây là ba build, mỗi build ba lượt truy vấn warm, không phải tốc độ Spark live hay phép đo cold cache.

The storage saving depends on what we count

Same RPKI experiment, now including the other retained files.

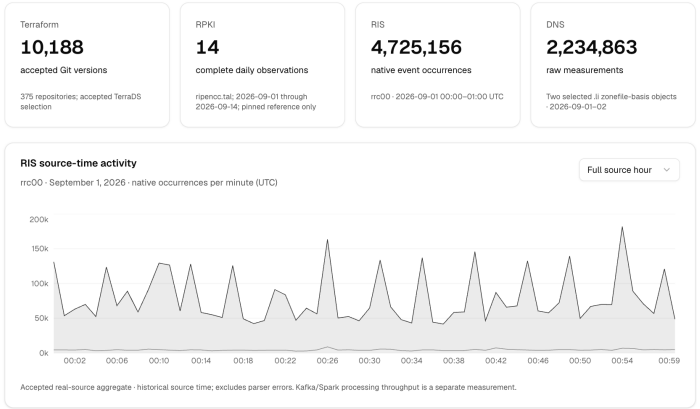


R2 saves 17.5% across measured local experiment files.
Sources, provenance maps and query scratch still occupy most of the space.



Khoảng 45 giây. Nếu chỉ nói tiết kiệm 92,6 phần trăm thì người nghe dễ hiểu thành toàn bộ hệ thống tiết kiệm như vậy. Khi cộng cả archive nguồn, ánh xạ về bản ghi gốc, metadata và file làm việc còn giữ lại, tổng file local giảm từ khoảng 251,98 MB xuống 207,90 MB, tức khoảng 17,5 phần trăm. Phần màu xanh đậm là dữ liệu phân tích đã tối ưu; phần xám vẫn lớn. Ý nghĩa thực tế: cần đo toàn bộ những thành phần đang giữ để quyết định tối ưu, không chỉ đo file đầu ra nhỏ nhất. Đây vẫn chỉ là tổng các thành phần của thí nghiệm biểu diễn local, chưa phải tổng disk của deployment vì không bao gồm Kafka, checkpoint hay overhead hệ thống file. Không suy kết quả RPKI thành mức tiết kiệm cho Terraform, RIS hoặc DNS.

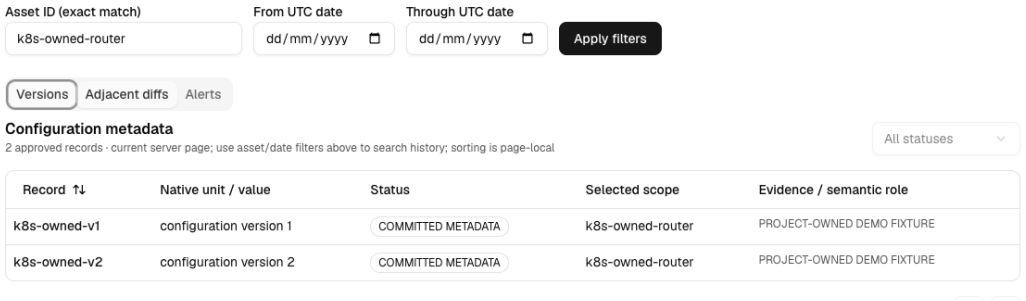
Dashboard: four domains in one evidence overview





Khoảng 30 giây. Đây là ảnh giao diện thật được mở từ frontend hiện tại. Bốn thẻ phía trên tương ứng Terraform, RPKI, RIS và DNS vừa giới thiệu. Biểu đồ bên dưới là số sự kiện RIS theo thời gian nguồn, không phải throughput xử lý của Spark. Phần lịch sử cấu hình có API để tra cứu theo asset và thời gian khi backend chạy. Các con số trên ảnh thuộc những đơn vị khác nhau, không cộng lại thành một chỉ số tổng. Ảnh này không chứng minh backend đang chạy trực tiếp trong buổi checkpoint.

Dashboard: inspect configuration history by asset



1. **Filter** by asset and UTC dates. 2. **Select** versions, adjacent diffs or alerts.
3. **Inspect** safe metadata from the configuration API.



Khoảng 35 giây. Trang lịch sử cấu hình có ba thao tác chính: lọc asset và ngày UTC, chọn tab versions hoặc adjacent diffs hoặc alerts, rồi xem metadata trả về từ API. Ảnh chụp là buổi rehearsal đã ghi nhận ngày 14 tháng 10, với asset do nhóm tạo có hai phiên bản và một diff. Con số TerraDS ở thẻ tổng quan là aggregate đã lưu, không có nghĩa toàn bộ lịch sử TerraDS được phục vụ qua API này. Trang history cần backend đang chạy; ảnh không chứng minh API hiện tại đang online.

Checkpoint readiness and next milestone

Requirement	What the group can show
Executable code	Runnable demo, domain pipelines and tests
Prepared data	Terraform, RPKI, RIS, DNS and source manifests
Architecture	Kafka, stateful Spark, committed lake and serving

Next: rehearse the latest integrated demo, repeat recovery checks, and extend workload measurements.

Existing deployment evidence is local / single-node. Scaling and HA remain open.

Thank you — questions?

Checkpoint readiness and next milestone

Requirement | What the group can show
Executable code | Runnable demo, domain pipelines and tests
Prepared data | Terraform, RPKI, RIS, DNS and source manifests
Architecture | Kafka, stateful Spark, committed lake and serving

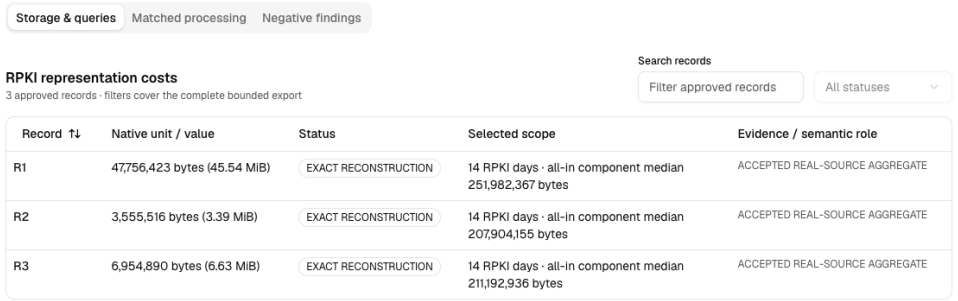
Next: rehearse the latest integrated demo, repeat recovery checks, and extend workload measurements.

Existing deployment evidence is local / single-node. Scaling and HA remain open.

Thank you — questions?

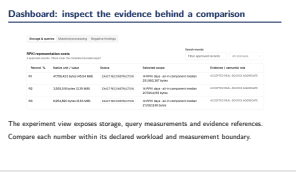
Khoảng 55 giây. Chốt lại ba yêu cầu của thầy: code chạy được, dữ liệu đã chuẩn bị, và kiến trúc đã có implementation cùng kết quả tích hợp. Nhóm đã chạy pipeline trước đây; bước tiếp theo là diễn tập lại trên image mới nhất, kiểm tra recovery và mở rộng phép đo. Không nói cả hệ thống production đã hoàn thành. Các cơ chế chính vừa được trình bày theo đường đi dữ liệu. Nếu cần xem thêm bảng thí nghiệm hoặc trạng thái vận hành trên dashboard thì mở hai slide phụ.

Dashboard: inspect the evidence behind a comparison



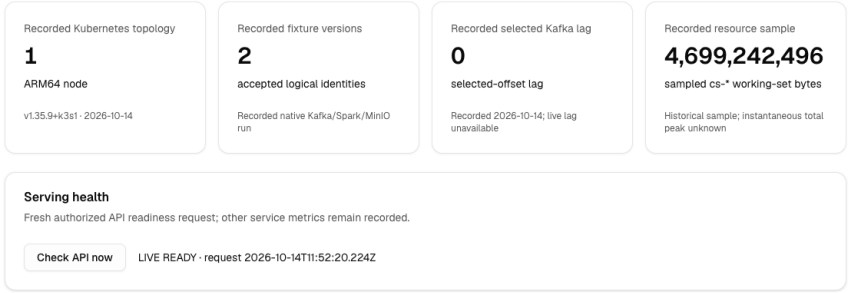
The experiment view exposes storage, query measurements and evidence references.

Compare each number within its declared workload and measurement boundary.



Slide phụ. Đây là màn hình experiments để đối chiếu kết quả với phạm vi và bằng chứng của phép đo. Bảng hiển thị R1, R2, R3 với byte phân tích và tổng component đã đo, có scope và bằng chứng. Các tab khác cho phép xem matched processing và negative findings. Đây là dữ liệu thí nghiệm đã được export và kiểm tra hash, không phải một job benchmark vừa chạy khi mở trang. Kết quả RPKI khớp hai biểu đồ trong phần trình bày chính.

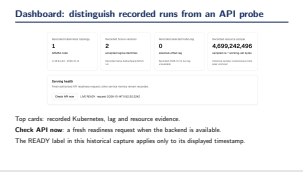
Dashboard: distinguish recorded runs from an API probe



Top cards: recorded Kubernetes, lag and resource evidence.

Check API now: a fresh readiness request when the backend is available.

The READY label in this historical capture applies only to its displayed timestamp.



Slide phụ. Ảnh có chữ LIVE READY nhưng đó là trạng thái của request tại timestamp trong ảnh, không phải trạng thái hiện tại. Các thẻ node, Kafka lag và resource là bằng chứng chạy đã lưu. Khi demo thật, bấm Check API now mới tạo request readiness và giao diện làm hết hiệu lực kết quả sau thời gian quy định. Readiness thành công chứng minh API và dependencies cần thiết cho request đó, không chứng minh toàn bộ pipeline đang ingest dữ liệu. Dashboard không có quyền đọc kubeconfig hay Kafka credentials và không phải một hệ thống monitoring cluster trực tiếp.